Node.js + Express & Handlebars

Desarrollo de Aplicaciones Web

Ingeniería de software

Introducción

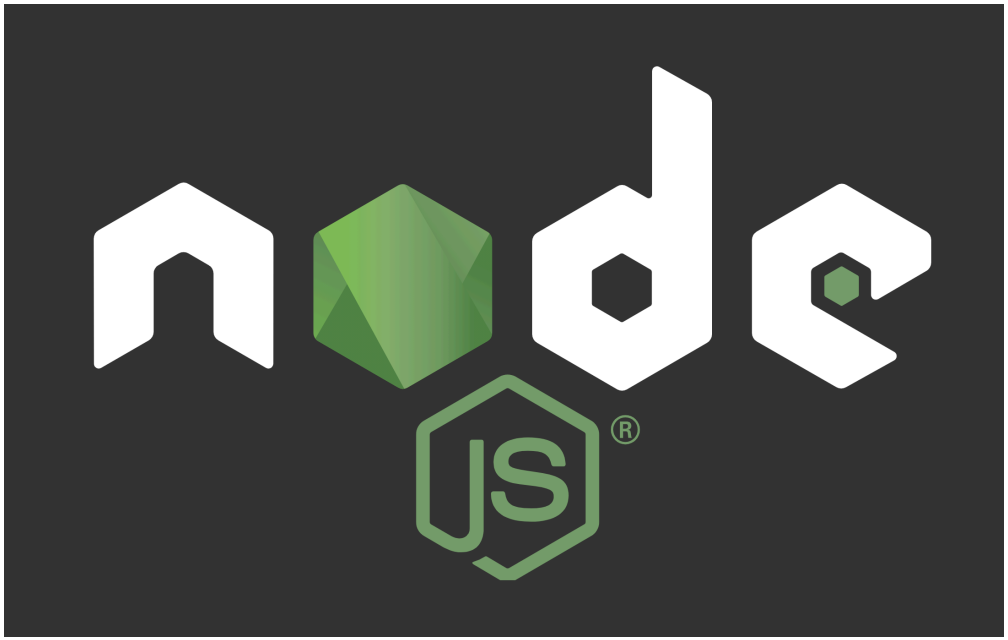
- Node.js
- Procesamiento de una petición
- Generación de una respuesta
- Acceso a MySQL
- Express
- Handlebars
- express-handlebars + handlebars-helpers

Node.js

- Plataforma para el desarrollo de aplicaciones basadas en el lenguaje Javascript.
  - Antes: Javascript era soportado solo por los navegadores.
  - Ej.: Ejecutar scripts Javascript en Node.js ">node miApliacion.js"
- Utiliza motor V8 Javascript (Chrome) e implementa ECMAScript 2015 (ES6).
- Lenguaje Javascript, no está disponible los objetos **document** y **window** del navegador.
- Node.js soporta módulos CommonJS y módulos ES.

Node.js

- https://nodejs.org/
  - Dirigido por Eventos.
  - Modelo de Entrada/Salida asíncrono.
  - Ligero y eficiente.



NPM

- Gestión de módulos y paquetes a través de **npm**.
  - *npm* no es acrónimo de "Node.js Package Manager" si no un retroacrónimo de "npm is not an acronym".
  - https://www.npmjs.com/.
  - Instala, comparte y distribuye librerías/módulos JS
  - Gestión de dependencias en proyectos JS.
- **npm** instala, actualiza y gestiona la descarga de dependencias de un proyecto.

NPM

- Las dependencias de un proyecto se definen en el archivo **package.json**.

```
npm install
```

- Los módulos o paquetes del repositorio *npm* se pueden instalar con:

```
npm install <nombre-paquete>@<version>
```

- Los módulos instalados se almacenan en la carpeta "node_modules" en el raiz del proyecto.

Módulo

- Node.js utiliza el concepto de módulo en su desarrollo.
  - Creación de librerías/módulos Javascript para componer una aplicación.

- Un módulo (CommonJS o ES module) contiene funciones, objetos y variables Javascript.
  - Permite indicar qué será exportado para ser utilizado por otros programas.

Módulo

- Ejemplo módulo *modulo.js*:

```
var PI=3.14;
function sumar(x1,x2){return x1+x2;}
function restar(x1,x2){return x1-x2;}
function dividir(x1,x2){
  if (x2==0) {mostrarErrorDivision();}
  else{ return x1/x2;}
}
function mostrarErrorDivision() {
  console.log('No se puede dividir por cero');
}
export {PI, sumar, restar};
export default dividir;
```

Módulo

- Se utiliza **export** para hacer accesible el contenido de un módulo.
- Se pude definir con **export default** para definir una variable, función u objeto accesible por defecto.
- Para utilizar los elementos exportados utilizamos **import**.

```
import dividir, {PI, sumar, restar} from './modulo.js';
var s = sumar(PI, PI);
var r = restar(PI, 3);
var d = dividir(s, 2);
```

Módulo

- Si no se especifica ruta, se busca en la carpeta "node_modules" del proyecto, y en la instalación de Node.
- Módulo **os** para acceder al sistema operativo.

```
import os from 'os';

console.log('Sistema operativo:'+os.platform());
console.log('Versión del OS:'+os.release());
console.log('Memoria total:'+os.totalmem()+' bytes');
console.log('Memoria libre:'+os.freemem()+' bytes');
```

Módulo

- Módulo **fs** para acceder al sistema de archivos.

```
fs.writeFile('./archivo.txt', 'linea 1\n',
    (err)=>{
        if(err) console.log(err);
        else console.log("Archivo creado");});
fs.appendFile('./archivo.txt', 'linea 2\n',
    (err)=>{
        if(err) console.log(err);
        else console.log("Archivo ya creado");});
fs.readFile('./archivo.txt',
    (err, datos)=>{
        if(err){console.log(err);}
        else console.log(datos.toString());});
console.log("Última línea de código");
```

Módulo

- **fs** implementa programación asincrónica para el manejo de ficheros.
- Resultado de la ejecución del script anterior:

```
Última línea de código
Archivo ya creado
Archivo creado
linea 1
linea 2
```

Servidor web en Node.js

- El módulo **http** tiene una función llamada *createServer* para crear un servidor web basado en el protocolo HTTP.

- *createServer* requiere una función con dos parámetros: **petición** y **respuesta**.

- Los objetos petición y respuesta son creados por la función *createServer* para dar acceso a la **petición** y **respuesta HTTP**.

Servidor web en Node.js

- Ejemplo creación servidor web:

```
import http  from 'http';
var servidor = http.createServer(
    function (peticion, respuesta) {
        respuesta.writeHead(200,
            {'Content-Type': 'text/html'});
        respuesta.write(
            '<!doctype html><html><head></head>'+
            '<body><h1>Hola Mundo</h1></body></html>');
        respuesta.end();
    });
servidor.listen(8080);
console.log('Servidor web iniciado');
```

Servidor web en Node.js

- *createServer()* se ejecuta y asocia un manejador de peticiones http de forma *asíncrona*.
  - **servidor.listen(8080)** es asíncrona y se queda esperando a recibir peticiones.

- El manejador de peticiones HTTP es la función pasada como argumento en *createServer()*.
  - En el ejemplo anterior, cualquier petición invocará a la función manejadora.

Servidor web en Node.js

- Escribir cabeceras en la respuesta HTTP.
  - **respuesta.writeHead**
- Escribir el contenido de la respuesta HTTP.
  - **respuesta.write**
- Indicar que se ha finalizado de escribir el contenido. **Obligatorio** para enviar respuesta.
  - **respuesta.end**
- Se puede invocar *respuesta.write* varias veces antes de *respuesta.end*.

Servidor web en Node.js

- Acceso al directorio raiz del servidor.

```
import { dirname } from "path";
import { fileURLToPath } from "url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);
```

MySQL

- Paquete oficial mysql2 es más moderno y compatible con Promises y async/await.
- Al usar async/await para manejar MySQL su implementación es más eficiente porque las transacciones no son bloqueantes.
- Instalación:

```
npm install mysql2
```

MySQL

- Creación y cierre de un pool de conexiones.

```
import mysql from "mysql2/promise";

const pool = mysql.createPool({
  host: "localhost",
  user: "root",
  password: "contraseña",
  database: "bbdd",
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
});

await pool.end();
```

MySQL

- Consulta de información de una base de datos.

```
const getUsers = async () => {
  try {
    const [rows, fields] = await pool.query("SELECT * FROM users");
    console.log(rows);
    fields.forEach((field) => {
      console.log(`Nombre: ${field.name}, Tipo: ${field.type}`);
    });
  } catch (error) {
    console.error("Error al obtener usuarios:", error);
  }
};

getUsers();
```

MySQL

- **rows** : array de objetos fila donde cada columna de la tabla tiene su correspondiente propiedad en el objeto.
  - rows[0].id
  - rows[0].nombre
- **fields**: array de definiciones de columna con los metadatos de las columnas de la tabla.
  - fields[0].name: nombre de la columna de bbdd.
  - fields[0].type: tipo de datos de la columna de la bbdd.

MySQL

- Añadir información en la base de datos.

```
const addUser = async (nombre) => {
  try {
    const [result] = await pool.query(
      "INSERT INTO users (nombre) VALUES (?)", [nombre]
    );
    console.log(`ID usuario: ${result.insertId}`);
  } catch (error) {
    console.error("Error al agregar usuario:", error);
  }
};

addUser("Jose");
```

MySQL

- Borrar filas de la base de datos.

```
const deleteUser = async (id) => {
  try {
    const [result] = await pool.query("DELETE FROM users WHERE id = ?", [id]);
    if (result.affectedRows > 0) {
      console.log(`Usuario con ID ${id} eliminado`);
    } else {
      console.log(`No se encontró el usuario con ID ${id}`);
    }
  } catch (error) {
    console.error("Error al eliminar usuario:", error);
  }
};

deleteUser(1);
```

MySQL

- Definir una transacción con los métodos *beginTransaction*, *query*, *commit*, *rollback* y *release*.

```
const addUsersTransaction = async (nombres) => {
  const conexion = await pool.getConnection();
  try {
    await conexion.beginTransaction();

    for(const nombre of nombres){
      await connection.query("INSERT INTO users (nombre) VALUES (?)", [nombre]);
    }

    await connection.commit();
  } catch (error) {
    await connection.rollback();
  } finally {
    connection.release();
  }
};

addUsersTransaction(["Jose", "Antonio"]);
```

Actualizar Node.js

- **n** gestiona tus versiones de Node.js de manera interactiva. https://www.npmjs.com/package/n

```
node -v
sudo npm install -g n
sudo n latest
```

Instalación express y handlebars

- Creación del esqueleto de la aplicación con el asistente **express-generator**, no requiere de instalación.

```
npx express-generator --view=hbs .
```

- Crea el proyecto en la carpeta actual.

Instalación express y handlebars

- Instalación del generador de aplicaciones de express y ejecución.

```
npm i -g express-generator
express --view=hbs aplicación
```

- Requiere de la instalación de dependencias en la carpeta de la aplicación.

```
npm install
```

¿Qué es Express.js?

- Express.js, también llamado «Express», es un framework de backend Node.js minimalista y rápido.
- Uno de los framework de backend más popular para Node.js, y es una parte extensa del ecosistema JavaScript.
- Proporciona características y herramientas robustas para desarrollar **aplicaciones web** de **backend JS** escalables.

¿Qué es Express.js?

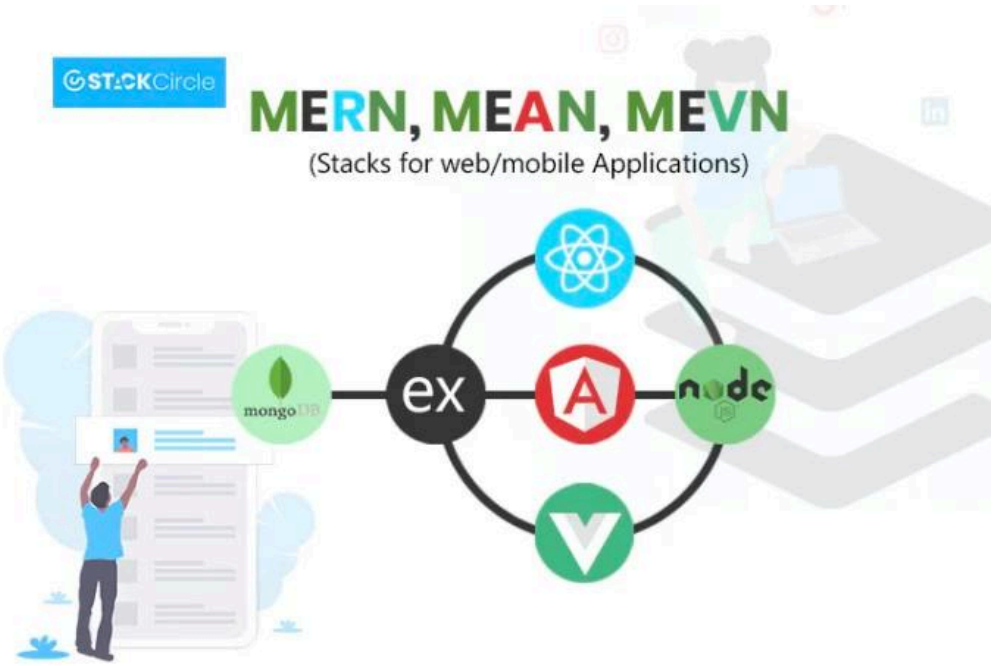
- Ofrece un **sistema de enrutamiento**, un sencillo manejo del modelo de **petición y respuesta** HTTP, middleware para construir y desplegar aplicaciones a gran escala, y otras características simplificadas para ampliar el framework.
  - Permite desarrollar aplicaciones basadas en endpoints API.
  - Favorece el desarrollo de una sola página (SPA): toda la aplicación se enruta en una sola página de índice.

Por qué usar Express.js

- Mayor velocidad de E/S para peticiones y respuestas rápidas.
- Sistema de hilo único.
- Procesos asíncronos.
- Basado en MVC para simplificar las manipulaciones de datos y los sistemas de enrutamiento.

Parte de cualquier Stack: MEAN, MERN, MEVN

- Framework de elección en cada stack representado con la E: MERN, MEAN, MEVN.
- Integra fácilmente cualquier tecnología, e.j. bases de datos.



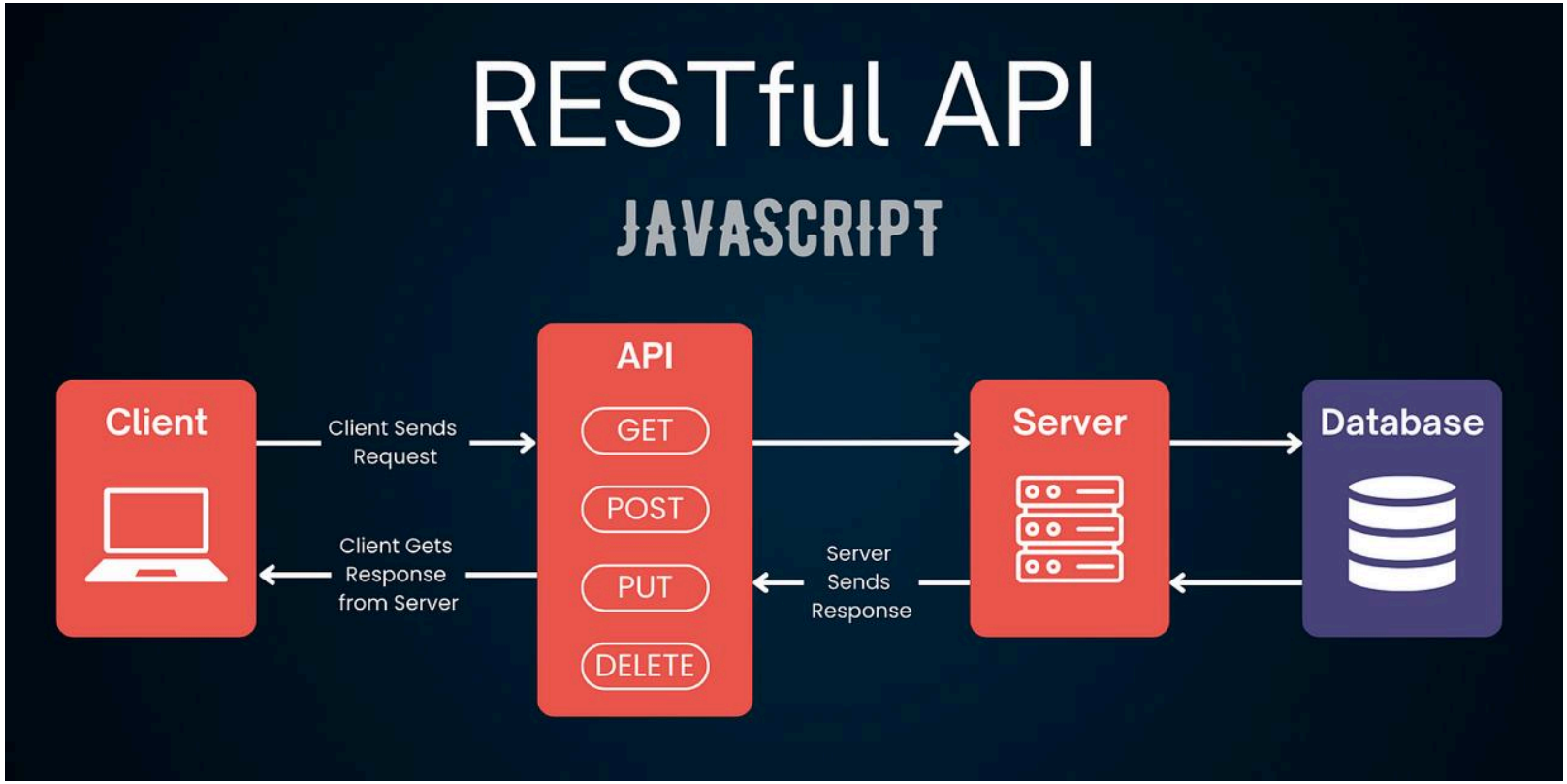
Sistema de Enrutamiento

El framework tiene un sistema de enrutamiento incorporado que asiste en la generación de una respuesta dada una solicitud del cliente, a través de un **endpoint** particular.

El sistema de enrutamiento permite organizar de forma sencilla los **endpoints** en diferentes archivos.

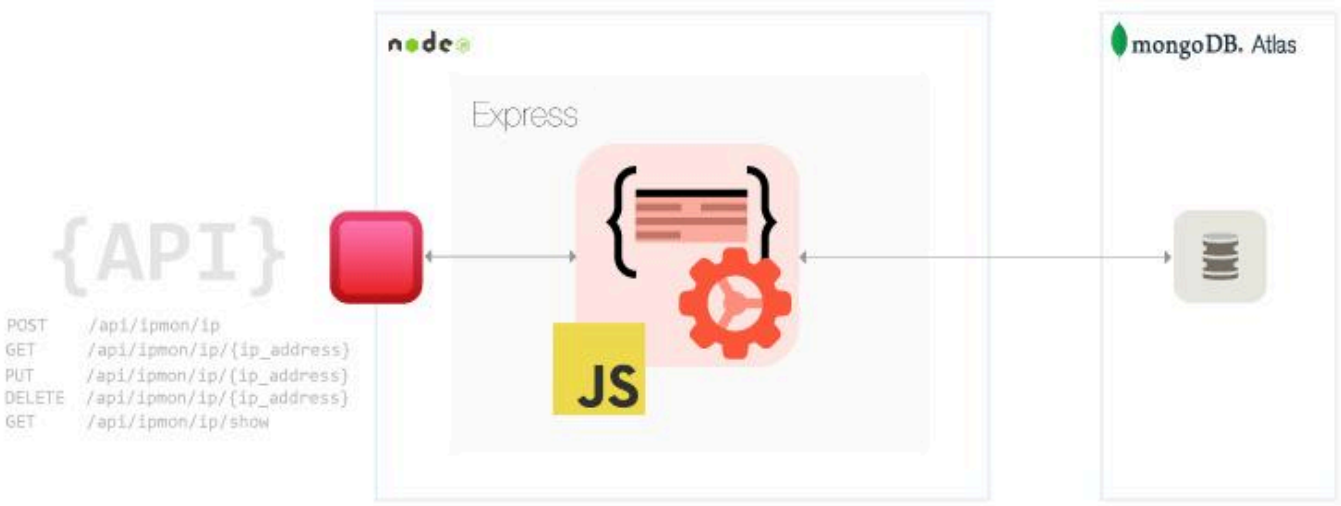
- Facilita la gestión de la estructura de la aplicación, agrupando diferentes rutas en una única carpeta/directorio.
- Creación de código más mantenibles, agrupando las funcionalidades con el enrutador y evitando la repetición.

Arquitectura API RESTful



Arquitectura API con Node + Express

- Enpoints API de accesso basados en Node y Express.



Cómo funciona Express

- Express se basa en el modelo cliente-servidor para aceptar las peticiones de los usuarios y devolver las respuestas al cliente.

- Funcionamiento no muy diferente del de otros frameworks webs.

- Cuando un usuario envía una petición desde el navegador, este envía una petición HTTP a la aplicación/servidor.

Cómo funciona Express

- El servidor recibe la petición a través de una de sus rutas y la procesa utilizando el manejador de petición que coincida con la ruta solicitada.

- Tras el procesamiento, el servidor enviará una respuesta HTTP al cliente.

- La respuesta devuelta al cliente puede ser un texto plano, una página HTML dinámica que el navegador procesará en una página web, o datos JSON, entre otros.

Ejemplo Express

Ejemplo simple de servidor Express que escucha peticiones entrantes en http://localhost:3000/ y devuelve una respuesta de texto plano de 'Hola Mundo'.

```
import express from 'express';
const app = express();
const port = 3000;
app.get('/', (request, response) => {
  response.send('Hola Mundo')
  })
app.listen(port, () => {
  console.log(`Test app listening at http://localhost:${port}`)
  })
```

Ejemplo RESTful API

```
import express from 'express';
const app = express();
const port = 3000;
app.listen(port, () => {
  console.log(`Test app listening at http://localhost:${port}`)})
const todos = [
    {         title: "TODO 1",
      desc: "Este es mi primer TODO",
      completed: true  },
    {         title: "TODO 2",
      desc: "Este es mi segundo TODO",
      completed: true  },
    {         title: "TODO 3",
      desc: "Este es mi tercer TODO",
      completed: true  },
    {         title: "TODO 4",
      desc: "Este es mi cuarto TODO",
      completed: true  },
    {         title: "TODO 5",
      desc: "Este es mi quinto TODO",
      completed: true  }
];
```

Ejemplo: GET

- A continuación, crearemos un endpoint para recuperar todos los `TODOS` almacenados en nuestro servidor:

```
app.get("/todos", (request, response) => {
  response.status(200).json(todos);
});
```

Ejemplo: GET

- A continuación, un endpoint para recuperar un único `TODO` basado en el `ID` del Todo:

```
app.get("/todos/:id", (request, response) => {
  response
    .status(200)
    .json({ data: todos.find((todo) => todo.id === request.params.id) });
});
```

Ejemplo: POST

- Ahora, un endpoint para almacenar un nuevo `TODO`:

```
app.post("/todos", (request, response) => {
  todos.push(request.body);
  response.status(201).json({ msg: "TODO creado correctamente." });
});
```

Ejemplo: PUT

- A continuación, un endpoint para actualizar un `TODO` existente con el `ID`:

```
app.put("/todos/:id", (request, response) => {
  const todo = todos.find((todo) => todo.id === request.params.id);

  if (todo) {
    const { title, desc, completed } = request.body;
    todo.title = title;
    todo.desc = desc;
    todo.completed = completed;
    response.status(200).json({ msg: "TODO actualizado correctamente." });
    return;
  }

  response.status(404).json({ msg: "TODO no encontrado." });

});
```

Ejemplo: DELETE

- Por último, crearemos un endpoint para eliminar un único `TODO` basado en el `ID`:

```
app.delete("/todos/:id", (request, response) => {
  const todoIndex = todos.findIndex((todo) => (todo.id === request.params.id));
  if (todoIndex) {
    todos.splice(todoIndex, 1);
    response.status(200).json({ msg: "TODO eliminado correctamente." });
    return;
  }
  response.status(404).json({ msg: "TODO no encontrado." });

});
```

- Este fragmento de código muestra cómo implementar una funcionalidad DELETE en Express.
- Recoge el ID de Todo a través de parámetros y busca en el array el ID que coincide y lo borra.

Explorar el Esqueleto de la Aplicación Express

La estructura básica que forma la columna vertebral de la aplicación web en Express creada con express-generator es (views= `hbs`):

```
/
|-- /node_modules
|-- /public
|-- /routes
    |-- index.js
    |-- users.js
|-- /views
    |-- error.hbs
    |-- index.hbs
    |-- layout.hbs
|-- app.js
|-- package.json
```

Estructura aplicación Express

- **node_modules:** Este directorio almacena todas las dependencias y librerías instaladas para el proyecto.
- **public:** Contiene activos estáticos como CSS, JavaScript, imágenes, etc. Estos archivos se sirven directamente al navegador del cliente.
- **views:** Contiene plantillas o vistas que el servidor renderiza para crear la interfaz de usuario. Aquí, error.hbs, index.hbs layout.hbs son plantillas escritas en el lenguaje de plantillas Handlebars. Ayudan a estructurar y mostrar contenido dinámico a los usuarios.

Estructura aplicación Express

- **routes:** Contiene los archivos responsables de definir varias rutas y gestionar las peticiones procedentes de diferentes URL.
- **app.js:** Este archivo suele servir como punto de entrada para la aplicación Express. Es donde se configura el servidor, se establece el middleware, se definen las rutas y se gestionan las peticiones y respuestas.
- **package.json:** Este archivo contiene metadatos sobre la aplicación. Ayuda a gestionar las dependencias y la configuración del proyecto.

Estructura aplicación Express

- Fichero **package.json**: configuración con scripts y dependencias (principalmente).
- Fichero **www** es el script de arranque del proyecto Express.
  - Levanta el servidor haciendo uso del módulo **http**.
  - **Puerto por defecto: 3000**, salvo que se indique otro en el parámetro de arranque **PORT**.
  - Hace uso del objeto express() creado en *app.js*.

Gestión de Rutas

- En una aplicación Express, el directorio `routes` es donde se definen las rutas como archivos independientes.
- La *ruta principal*, a menudo llamada ruta raíz, reside en el archivo *routes/index.js*.
- Esta ruta raíz se encarga de una solicitud **GET**, respondiendo con una página web generada en HTML por el framework.

Gestión de rutas

- A continuación se muestra un fragmento de código que ilustra cómo se maneja una solicitud **GET** para renderizar una página de bienvenida básica.

```
import express from 'express';
const router = express.Router();

router.get('/', function(req, res, next) {
    res.render('index', { title: 'Express' });
});

export router;
```

Gestión de rutas

- Si se modifica la función *res.render* por *res.send*, el tipo de respuesta cambia de **HTML** a **JSON**:

```
import express from 'express';
const router = express.Router();

router.get('/', function(req, res, next) {
    res.send({ key: 'value' });
});

export router;
```

Gestión de rutas

- Ampliando las capacidades, se añade otra ruta al mismo archivo, introduciendo un nuevo endpoint que acepta un parámetro.

```
router.get('/newEndpoint', function(req, res, next) {
  res.send({ yourParam: req.query.someParam });
});
```

- El envío de una solicitud *GET* a *localhost:3000/newEndpoint?someParam=whatever* producirá una salida JSON que contendrá la cadena *whatever*.

Procesamiento de datos en la Request

- Los datos de una petición (**request**) en HTTP pueden venir:
- **request.query**
  - En la URL de la petición, como parámetros HTTP: se incluyen los datos tras el símbolo **?** y los datos vienen con la forma **nombre_parametro=valor_parametro**, siendo todos los datos de tipo *string*.

Procesamiento de datos en la Request

- **request.body**
  - En el cuerpo de la petición: no se ven en la URL y habitualmente se usan en request de tipo *POST* y *PUT*. Mecanismo para envío de datos binarios (ficheros)
- **request.params**
  - Como parte de la URL (path) de la petición: el path de la URL incluye los parámetros. Mecanismo para endpoints **RESTful**.

Ejemplo:

```
//extended: false, parsear solo parámetros string
app.use(express.json());
app.use(express.urlencoded({ extended:true }));

router.get('/users', function(req, res, next) {
    let nombre = req.query.nombre;
    res.status(200).send(`<p>GET ${nombre}<p>`)
})
router.post('/enviardatos', function (req, res) {
    let id = req.body.id;
    res.status(200).send(`<p>POST ${id}<p>`)
})
router.get('/users/:id', function(req, res, next) {
    let id = req.params.id;
    res.status(200).send(`<p>GET ${id}<p>`)
})
```

Respuesta con contenido estático

- Configuración de la aplicación Express: **app.use()**.

- Configura la carpeta de las páginas estáticas de Express (html + css + js para el navegador) con **express.static**:

```
app.use(express.static(path.join(__dirname, 'public')));
```

Respuesta con contenido estático

- **path.join()** : permite construir un path, concatenando varias partes del path (se pueden usar múltiples argumentos)
  - Por ejemplo:

```
app.use(express.static(path.join(__dirname, 'views', 'partials')));
```

- **__dirname** : hace referencia al directorio de la aplicación en node+express.

Resumen de Express

- Express es el framework más popular del ecosistema Node.js, y no es difícil ver por qué.
- Ofrece un amplio abanico de ventajas y características.
- La escasa curva de aprendizaje hace que sea muy sencillo y fácil de usar.
- Abstrae las funciones innecesarias o no deseadas de la aplicación web y te proporciona una fina capa de funciones básicas que permiten la flexibilidad.

Resumen de Express

- Se basa en Nodejs, lo que supone que conocer Express favorece conocer otros frameworks populares.
- Este conocimiento puede ayudar a tomar decisiones sobre la construcción de la lógica del negocio y qué otros frameworks utilizar junto a Express.

Handlebars: un motor de plantillas en Express

- Además de las funciones de **enrutamiento** de Express, es necesario destacar que el framework puede utilizar un **lenguaje de plantillas dinámico** para la **generación de páginas web**.
- Las plantillas (layouts) están por defecto en la carpeta **/views**.
    - Se puede cambiar esta configuración.

Handlebars

- Es posible configurar la ubicación de las plantillas:

```
app.set('views', path.join(__dirname, 'views'));

app.set('view engine', 'hbs');
```

- En el ejemplo anterior se configura el motor de plantillas `hbs` (**Handlebars**), aunque no es el únic motor de plantillas actualmente (`jade` es otro muy popular).

Layout de las plantillas .hbs

- Se puede especificar qué layout se desea usar como parte de la llamada de renderizado render().
- Se pueden **usar varios layouts**
  - Si se crea un nuevo diseño llamado **otro_layout.hbs**, en la carpeta **views**:

```
res.render('una_plantilla',
        { title: 'my other page', layout: 'otro_layout' });
```

- Se puede cambiar el layout defecto en **app.js**:

```
app.set('view options', { layout: 'otro_layout' });
```

Handlebars

- Basado en el sistema de plantillas **Moustache**.
- Es un lenguaje de plantillas simple.
- Utiliza una **plantilla** y un **objeto de entrada (datos)** para generar HTML u otros formatos de texto.
- Las plantillas parecen texto normal con expresiones de Handlebars incrustadas.

```
<p>{{firstname}}{{lastname}}</p>
```

Handlebars

- Una expresión en Handlebars es **{{** *algunos contenidos* seguidos de **}}**.
- Cuando se ejecuta la plantilla, estas expresiones se reemplazan (**renderiza**) con valores/datos de un objeto de entrada.
- Fuera de Express, el motor de plantillas de Handlebars se puede ejecutar en el navegador con JS.
- Compilador de plantillas Handlebars: *Handlebars.compile*.

Handlebars

```
<!-- Include Handlebars from a CDN -->
<script src="https://cdn.jsdelivr.net/
  npm/handlebars@latest/dist/handlebars.js"></script>
<script>
  // compile the template
  var template =
  Handlebars.compile("Handlebars <b>{{doesWhat}}</b>");
  // execute the compiled template and
  // print the output to the console
  console.log(template({ doesWhat: "rocks!" }));
</script>
```

Expresiones Sencillas

- Plantilla:

```
<p>{{firstname}}{{lastname}}</p>
```

- Entrada:

```
{
  firstname: "Yehuda",
  lastname: "Katz",
}
```

- Salida:

```
<p>Yehuda Katz</p>
```

Expresiones Anidadas

```
{
  person: {
    firstname: "Yehuda",
    lastname: "Katz",
  },
}
```

```
{{person.firstname}}{{person.lastname}}
```

Expresiones en un contexto

- Block-helper **with**

```
{{#with person}}
{{firstname}} {{lastname}}
{{/with}}
```

```
{
  person: {
    firstname: "Yehuda",
    lastname: "Katz",
  },
}
```

Expresiones en un contexto

- Block-helper **each**

```
<ul class="people_list">
  {{#each people}} <li>{{this}}</li> {{/each}}
</ul>
```

```
{  people: [ "Yehuda Katz",
    "Alan Johnson",
    "Charles Jolley", ],
}
```

```
<ul class="people_list">
    <li>Yehuda Katz</li>
    <li>Alan Johnson</li>
    <li>Charles Jolley</li>
</ul>
```

Comentarios en la plantilla

```
{{! This comment will not show up in the output}}
<!-- This comment will show up as HTML-comment -->
{{!-- This comment may contain mustaches like }} --}}
```

- Cualquier comentario que deba contener **}}** u otro handlebars token, entonces se debe usar la sintaxis **{{!-- --}}**.

Expresiones de bloque (Helpers) propias

- Declaración de un block-helper **list**.

```
{{#list people}}{{firstname}} {{lastname}}{{/list}}
```

```
Handlebars.registerHelper("list", function(items, options) {
  const itemsAsHtml = items.map(item => "<li>" + options.fn(item) + "</li>");
  return "<ul>\n" + itemsAsHtml.join("\n") + "\n</ul>";
});
```

Expresiones de bloque (Helpers) propias

```
{  people: [
  { firstname: "Yehuda",
    lastname: "Katz"},
  { firstname: "Carl",
    lastname: "Lerche"},
  ]
```

```
<ul>
  <li>Yehuda Katz</li>
  <li>Carl Lerche</li>
</ul>
```

Uso de Partials

- El uso de **Partials** `{{>partial }}` permite la *reutilización del código mediante la creación de plantillas compartidas*.

```
Handlebars.registerPartial(
    "person",
    "{{person.name}} is {{person.age}} years old.\n"
)
```

Uso de Partials

```
{{#each persons}}
  {{>person person=.}}
{{/each}}
```

```
{  persons: [{ name: "Nils", age: 20 },
    { name: "Teddy", age: 10 }]
}
```

```
Nils is 20 years old.
Teddy is 10 years old.
```

Partials como plantillas HBS

- Se pueden usar plantillas HBS como partial y se pueden pasar argumentos.
- En una plantilla HBS:

```
{{> foot autor='Jose A. Miñarro Giménez' }}
```

- En el proyecto:

```
/views/partials/foot.hbs
```

Partials como plantilla HBS

- Para que esto funcione es necesario leer los ficheros partials y registrarlos, como si fuera una función.
- Más **sencillo** con el uso de la librería **express-handlebars**.

```
npm install --save express-handlebars`
```

Helpers para Handlebars + Express

- npm package: **handlebars-helpers** (https://www.npmjs.com/package/handlebars-helpers).
- Más de 130 helpers en ~20 categorías.
- Categorías: array, code, collection, comparison, date, fs, html, i18n, inflection, logging, markdown, match, math, misc, number, object, path, regex, string, url

```
npm install --save handlebars-helpers`
```

Ejemplo handlebars-helpers

```
var helpers = require('handlebars-helpers')();
//=> returns object with all (130+) helpers
```

```
var helpers = require('handlebars-helpers')(['math', 'string']);
//=> only the math and string helpers
```

```
var helpers = require('handlebars-helpers');
var math = helpers.math();
//=> only the math helpers
var helpers = require('handlebars-helpers');
var array = helpers.array();
//=> only the collections helpers
```

Ejemplo handlebars

- **express-handlebars** ofrece importantes facilidades de configuración del motor de Handlebars (**engine**) y definición de **Helpers** de apoyo.

```
const hbs = require('express-handlebars')
const hbshelpers = require('handlebars-helpers')
const multihelpers = hbshelpers()
```

Ejemplo handlebars

- Nótese que se define rutas para 'layouts' y 'partials':

`layoutsDir`, `partialsDir`

```
//Commment out => app.set('views', path.join(__dirname, 'views'))
app.engine(  "hbs", hbs.engine({
  helpers: multihelpers,
  extname: ".hbs",
  partialsDir: ["views/partials"],
  layoutsDir: "views",
  defaultLayout: "layout"}));
```

Ejemplo handlebars

- Ofrece también la posibilidad de configurar el layout por defecto: **defaultLayout**.
- Por útlimo (aunque hay más configuraciones), que permite declarar helpers propios o de terceros, de forma sencilla: **helpers**.
- Ejemplo de helpers propios y uso selectivo de helpers de terceros: **handlebars-helpers**.
  - En el ejemplo, importación selectiva de los helpers *object* y *string* de *handlebars-helpers*.
  - Definición de helper propio: *helpersFS*.
  - Configuración del engine de Handlebars con los dos previos helpers:

Ejemplo handlebars

```
const hbs = require('express-handlebars');
const hbshelpers = require('handlebars-helpers');
const multihelpers = hbshelpers(['object', 'string']);
//const multihelpers = hbshelpers();

const helpersFS = {
  hlp: echo => `Echo: ${echo}.`,
  STATIC: `/static`,
};

app.engine(
  "hbs",
  hbs.engine({
    helpers: {...multihelpers, ...helpersFS},
    partialsDir: ["views/partials"],
    extname: ".hbs",
    layoutsDir: "views",
    defaultLayout: "layout"
  })
);
```

Uso de Helper y Partial

- En un layout hbs, usar:

```
HELPER </br>
    {{hlp 'bar' }}
    {{split "a,b,c" ","}}
PARTIAL  </br>
    {{> foot autor='Jose A Miñarro Giménez' }}
```

- **hlp**: es el helper propio construido con la facilidad de *express-handlebars*.
- **split**: es el helper importado de *string* de *handlebars-helpers*.
- **foot**: es una plantilla dentro del directorio de *partials*.

Partials con parámetros

- Layout:

```
{{> myPartial parameter=favoriteNumber }}
```

- Partial : *myPartial*

```
The result is {{parameter}}
```

- Datos:

```
{ favoriteNumber: 123 }
```

Block-Helpers predefinidos (built-in)

- Iteradores, condicionales, with,...
- Usar **if** helper para renderizar condicionalmente un bloque. Si el argumento devuelve false, undefined, null, "", 0, ó [], Handlebars no renderizará el bloque.

```
{{#if author}}
  <h1>{{firstName}} {{lastName}}</h1>
{{/if}}
```

Block-Helpers predefinidos (built-in)

- Usar **unless** helper a la inversa del helper *if*. El bloque se renderiza si la expresión devuelve un valor de falsedad.

```
<div class="entry">
  {{#unless license}}
    <h3 class="warning">WARNING: This entry does not have a license </h3>
  {{/unless}}
</div>
```

Block-Helpers predefinidos

- Iterar sobre una lista usando **each** helper.
- Dentro del bloque se puede usar la referencia **this** para referirise a cada elemento de la iteración.

```
<ul class="people_list">
  {{#each people}}
    <li>{{this}}</li>
  {{/each}}
</ul>
```

Block-Helpers predefinidos

- Se puede usar la sección **else** y hacer referencia al índice del array con **{{@index}}**.

```
{{#each paragraphs}}
  <p>{{this}}</p>
{{else}}
<p class="empty">No content</p>
{{/each}}
```

```
{{#each array}} {{@index}}: {{this}} {{/each}}
```

Block-Helpers predefinidos

- El helper **with** permite cambiar el contexto de evaluación de una parte de la plantilla.

```
{{#with person}}
  {{firstname}} {{lastname}}
{{/with}}
```

```
{
  person: {
    firstname: "Yehuda",
    lastname: "Katz",
  },
}
```

ANEXO:

Handlebars: Access has been denied to resolve the property "from" because it is not an "own property" of its parent

(https://stackoverflow.com/questions/59690923/handlebars-access-has-been-denied-to-resolve-the-property-from-because-it-is)

Instalar la dependencia:

```
npm i @handlebars/allow-prototype-access
```

ANEXO

```
const Handlebars = require('handlebars');
```

```
app.engine('hbs', hbs.engine({
    defaultLayout: 'main',
    // ...implement newly added insecure prototype access
    handlebars: allowInsecurePrototypeAccess(Handlebars)
    })
);
```